

SMU Lyle School of Engineering

Reinforcement Learning

Policy Gradient Methods

Sutton/Barto* Chapter 13

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018



Online Material

CC BY NC SA

Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - Multi-step bootstrapping
  - Planning and learning with tabular methods
- **Part II: Approximate Solution Methods**
  - Prediction and Control using Approximation
  - Eligibility Traces
  - **Policy Gradient Methods**
- Part III: Modern RL Methods
  - Deep Reinforcement Learning
  - Current Applications

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a.
$p(s' \mid s, a)$	probability of transition to state s' fom state s taking action a.
$r(s, a)$	expected immediate reward from state s after action a.
$r(s, a, s')$	expected immediate reward from state s to s' with action a.
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Approximation

$\mu(s)$	on policy distribution (fraction of time spent in state)
α	step size (gradient descent learning rate)
$\hat{v}(s, \boldsymbol{w})$	approximate value function
$\hat{q}(s, a, \boldsymbol{w})$	approximate action-value function
$\pi(a \mid s, \boldsymbol{\theta})$	approximate policy
$\widehat{VE}$	mean squared value error
$\nabla f(x)$	gradient of function f evaluated at x

Parametrized Policies

Learn approximate policies

Learn a Parameterized Policy

- **Up to now**: Learn parameters for an approximate action-value function $\hat{q}$ and then extract a greedy policy. We still have been using a tabular policy representation.

- **New idea**: Use a policy-based method that learn parameters for a policy.

  Parameterized stochastic policy:

$$\pi(a|s, \boldsymbol{\theta}) = \Pr\{A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}\}, \qquad \boldsymbol{\theta} \in \mathbb{R}^d$$

- Learn $\boldsymbol{\theta}$ given a performance measure $J(\boldsymbol{\theta})$ using **approximate gradient ascent**.

  Update rule:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}$$

  where $\widehat{\nabla J(\boldsymbol{\theta}_t)} \in \mathbb{R}^d$ is a stochastic estimate of the gradient of the performance measure with respect to $\boldsymbol{\theta}$. We use an estimate because getting the exact gradient is hard.

Policy Gradient Theorem

- We define the performance measure as the true value of following π_θ from the start state s_0

$$J(\boldsymbol{\theta}) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s_0\right]$$

- The **policy gradient theorem** gives us an analytical expression for the unbiased gradient of the performance with respect to the policy parameters:

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\boldsymbol{\theta})$$

On-policy distribution under π and q_π represents the expectation over returns.

Gradient of the policy approximation function.

- **Important:**
  - No knowledge of the model (transition probabilities, etc.) is needed.
  - $\mu(s)$ means that sample access is sufficient.

Optional: Proof of the Policy Gradient Theorem

Proof of the Policy Gradient Theorem (episodic case)

With just elementary calculus and re-arranging of terms, we can prove the policy gradient theorem from first principles. To keep the notation simple, we leave it implicit in all cases that π is a function of $\boldsymbol{\theta}$, and all gradients are also implicitly with respect to $\boldsymbol{\theta}$. First note that the gradient of the state-value function can be written in terms of the action-value function as

$$
\begin{aligned}
\nabla v_\pi(s) &= \nabla \left[\sum_a \pi(a|s) q_\pi(s,a) \right], \quad \text{for all } s \in \mathcal{S} && \text{(Exercise 3.18)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla q_\pi(s,a) \Big] && \text{(product rule of calculus)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \nabla \sum_{s',r} p(s',r|s,a)\big(r + v_\pi(s')\big) \Big] \\
& && \text{(Exercise 3.19 and Equation 3.2)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) \nabla v_\pi(s') \Big] && \text{(Eq. 3.4)} \\
&= \sum_a \Big[\nabla \pi(a|s) q_\pi(s,a) + \pi(a|s) \sum_{s'} p(s'|s,a) && \text{(unrolling)} \\
&\qquad \sum_{a'} \big[\nabla \pi(a'|s') q_\pi(s',a') + \pi(a'|s') \sum_{s''} p(s''|s',a') \nabla v_\pi(s'') \big] \Big] \\
&= \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \to x, k, \pi) \sum_a \nabla \pi(a|x) q_\pi(x,a),
\end{aligned}
$$

after repeated unrolling, where $\Pr(s \to x, k, \pi)$ is the probability of transitioning from state s to state x in k steps under policy π. It is then immediate that

$$
\begin{aligned}
\nabla J(\boldsymbol{\theta}) &= \nabla v_\pi(s_0) \\
&= \sum_s \left(\sum_{k=0}^{\infty} \Pr(s_0 \to s, k, \pi) \right) \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_s \eta(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(box page 199)} \\
&= \sum_{s'} \eta(s') \sum_s \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_a \nabla \pi(a|s) q_\pi(s,a) \\
&= \sum_{s'} \eta(s') \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Eq. 9.3)} \\
&\propto \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s,a) && \text{(Q.E.D.)}
\end{aligned}
$$

Parametrized Policy Example

Requirements

- $\pi(a|s, \boldsymbol{\theta})$ needs to be **differentiable** with respect to $\boldsymbol{\theta}$ so that $\nabla\pi(a|s, \boldsymbol{\theta})$ exists and is finite.
- π needs to stay soft to guarantee **exploration** and convergence.

Popular for a small, discrete action space: **Soft-max in action preferences**

1. Define a **numerical preference** function where you prefer state-action combinations with larger scores.

$$h(s, a, \boldsymbol{\theta}) \in \mathbb{R}$$

   Some options:
   - Linear approximation of state preferences: $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{x}(s, a)$
   - ANN with $\boldsymbol{\theta}$ as network weights.

2. Giving the action with the highest preference in each state the highest probability is done using **soft-max**:

$$\pi(a|s, \boldsymbol{\theta}) = \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

Note: Soft-max converts a set of scores into a probability distribution.

Advantages Over ϵ-Greedy Action Selection

- **Learns stochastic policies**
  - The **stochastic policy will be driven to the optimal policy.** It can become a deterministic policy.
  - Stochastic policies can **perform exploration implicitly**. The amount of exploration is automatically reduced when it converges to the best action (deterministic policy).

    Note: Value-action methods cannot learn stochastic policies and are often stuck with learning a soft ϵ-greedy policy!

- **Smooth updates**
  - Parameterized policy changes action probability smoothly, while ϵ-greedy policies jump when the best action changes.

- **Incorporating prior knowledge is easier**
  - Prior knowledge about the form of the policy can be incorporated into the way the policy is parameterized (i.e., functional form of the preference function).
    This can improve learning significantly.

REINFORCE

Monte Carlo Policy Gradient Control

REINFORCE Update

Normal stochastic gradient-ascent algorithm update: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \nabla J(\boldsymbol{\theta}_t)$

Policy Gradient Theorem

- $\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\boldsymbol{\theta})$

$$= \mathbb{E}_\pi \left[\sum_a \pi(A_t|S_t,\boldsymbol{\theta}) q_\pi(S_t,a) \frac{\nabla \pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})} \right]$$

(replace s by the sampled $S_t \sim \mu$)

(replace a by the sampled $A_t \sim \pi$)

$$= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})} \right]$$

(because $\mathbb{E}_\pi[G_t|S_t,A_t] = q_\pi(S_t,A_t)$)

$$= \mathbb{E}_\pi[G_t \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})]$$

(because $\nabla \ln x = \frac{\nabla x}{x}$)

A_t, S_t and an unbiased estimate of G_t under the current π can be obtained via MC.

Convergence to a local optimum:
- linear approximation
- step size reduction

REINFORCE Algorithm

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Policy approximation is soft, so it explores!

For linear preferences

$$\pi(a|s,\boldsymbol{\theta}) = \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

$$h(s,a,\theta) = \theta^\top \boldsymbol{x}(s,a)$$

Gradient:

$$\nabla \ln \pi(a|s,\boldsymbol{\theta}) = \boldsymbol{x}(s,a) - \sum_b \pi(b|s,\boldsymbol{\theta}) \boldsymbol{x}(s,b)$$

Issue:

- MC sampling needs the whole episode and leads to high variance in the gradient estimate (which is even worse than the variance in sampled episode returns).

Convergence: Typically converges to a local optimum.

Code example:

REINFORCE: Monte Carlo Control with Policy Approximation for the L-Maze

See Code on https://michael.hahsler.net/Introduction_to_Reinforcement_Learning/

REINFORCE With Baseline

REINFORCE: Monte-Carlo Policy-

Input: a differentiable policy parameteriz
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g.,

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S$
  Loop for each step of the episode $t =$
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

REINFORCE with Baseline (episodic), for estimating π π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Used as the baseline

Variance reduction:

- The return is replaced by $G_t - b(S_t)$ this reduced MC's variance but introduces a bias.
- A popular choice is $b(S_t) = \hat{v}(S_t)$. The letter δ is used in the code since it represents an error.

Convergence: Typically converges to a local optimum.

Actor-Critic Methods

Combining an

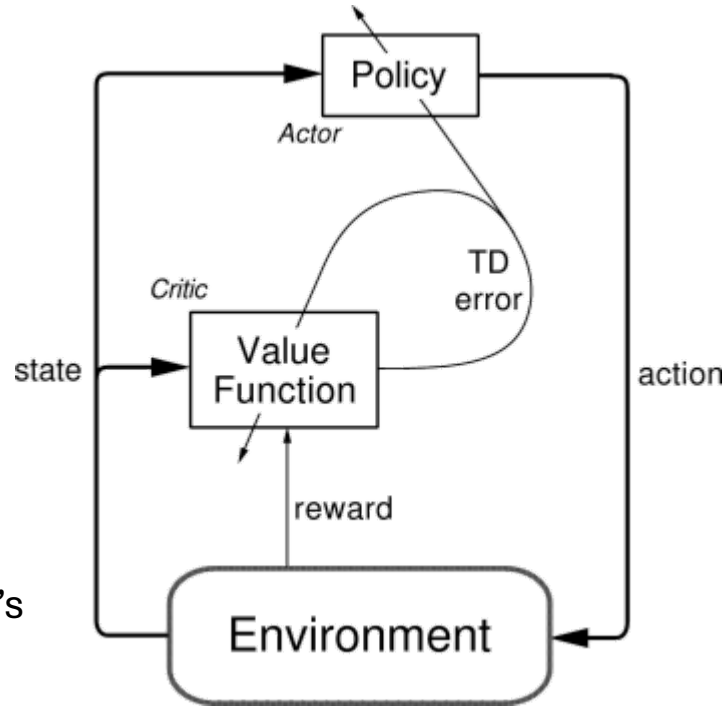
- actor (a parameterized policy) with a
- critic (using a parameterized value functions)

Actor-Critic Methods

Idea: estimate **a parameterized policy + a parameterized value function**

- **Actor**: estimates a parameterized policy $\pi(a|s, \boldsymbol{\theta})$ and decides on actions.
- **Critic**: evaluates the actor's decisions with an estimate of the TD error using its own value function estimate for the current policy $V = \hat{v}_\pi(s, \boldsymbol{w})$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$ 1-step version



Updates

- The critic uses the TD error to update its value function estimate.
- The actor uses the TD error as the actor's judgment of its last action:
  - **Positive TD error**: the last action performed better than expected, and the actor's tendency to pick it should be strengthened.
  - **Negative TD error**: suggests the actor's tendency should be weakened.

The TD errors acts as an **implicit baseline for the actor** which reduced variance.

Advantages:

- The update no longer uses MC as in REINFORCE.
- The baseline contained in the TD error reduces variance and improves learning.

The Policy Gradient Update Signal

- True policy gradient depends on $q_\pi(s,a)$ as the update signal

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\boldsymbol{\theta}) = \mathbb{E}_\pi \left[\sum_a q_\pi(s,a) \nabla \pi(a|s,\boldsymbol{\theta}) \right]$$

- The update can use any signal X

$$\nabla J(\boldsymbol{\theta}) \propto \mathbb{E}_\pi \left[\sum_a X \, \nabla \pi(a|s,\boldsymbol{\theta}) \right]$$

  as long as $\mathbb{E}_\pi[X|s,a] \propto \mathbb{E}_\pi[q_\pi(s,a)]$, i.e., if it tells us how good an action is.

- Examples of valid signals:
  - MC (REINFORCE): $X = G_t$
  - Advantage function: $X = A(s,a) = q(s,a) - v(s)$
  - TD error: $X = \delta_t = R_{t+1} + \gamma v(S_{t+1}) - v(S_t)$

Advantage: Measures how much better a is compared with the current action in $\pi(s)$.

One-Step Actor-Critic Algorithm

REINFORCE with Baseline (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiabl
Algorithm parameter
Initialize policy p

Loop forever (for ea
Generate an episo
Loop for each step
$G \leftarrow \sum_{k=t+1}^{T} \gamma$
$\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta$
$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \gamma^t \delta$

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$ — actor
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$ — critic
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Discounting multiplier: $\mathrm{I} = \gamma^t$

Policy approximation is soft, so it explores!

TD error: using $\hat{v}$ estimates.

Update the critic (value function) = $TD(0)$

Update the actor (policy) using the TD error signal.

Code example:

Actor-Critic: L-Maze

See Code on https://michael.hahsler.net/Introduction_to_Reinforcement_Learning/

Off-Policy Learning with Policy Gradient Methods

- We have only talked about on-policy methods:

  This is fine since the **parameterized policy can approximate the optimal deterministic policy**.

- **Issue**: We cannot use data collected using a different policy! This is a problem for sample efficiency, especially in deep RL.

- **Off-policy Actor-Critic methods** exist and are typically based on neural networks:
  - Deep Deterministic Policy Gradient (DDPG)
  - Twin Delayed DDPG (TD3)
  - Soft Actor Critic (SAC)

Continuous Actions

Parameterized policies can represent continuous actions.

Policy Parameterization for Continuous Actions

Issue: Soft-max in action preferences works well for a small number of discrete actions.

For continuous actions: Choose action from a continuous distribution and learn the distribution parameters.

Example: Choose the action (value) from a Gaussian distribution:

$$\pi(a|s,\boldsymbol{\theta}) = \frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{\left(a-\mu(s,\boldsymbol{\theta})\right)^2}{2\sigma(s,\boldsymbol{\theta})^2}\right)$$

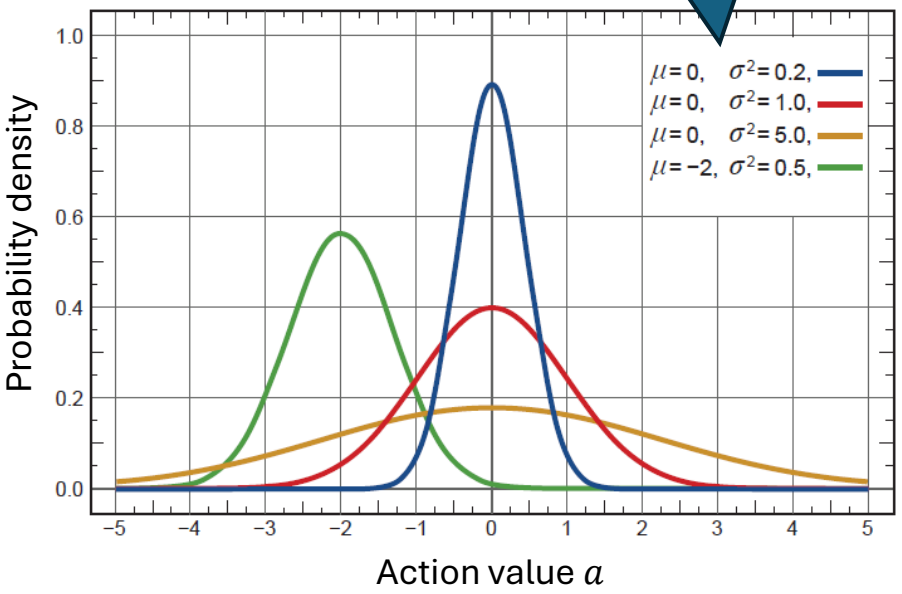
where often

$$\mu(s,\boldsymbol{\theta}) = \boldsymbol{\theta}_\mu^\top \boldsymbol{x}_\mu(s)$$
$$\sigma(s,\boldsymbol{\theta}) = \exp(\boldsymbol{\theta}_\sigma^\top \boldsymbol{x}_\sigma(s))$$

is learned with $\boldsymbol{\theta} = [\boldsymbol{\theta}_\mu, \boldsymbol{\theta}_\sigma]$

Linear in state features.

Depend on state and learned parameters θ



Value Function Approximation vs. Policy Gradient Methods

Value-Function Approximation

Advantages

- **Deterministic / Greedy Clarity:** Directly maximizing $Q(s, a)$ leads to a greedy, deterministic policy.
- **Established Off-Policy Learning:** Well-suited for temporal-difference (TD) learning, allowing agents to learn from arbitrary or historical transition experiences.
- **Sample Efficiency:** Off-Policy learning from old data is easier.
- **Global optimum** for some settings.

Disadvantages

- **Instability with Function Approximation:** Combining bootstrapping (TD learning), off-policy training, and nonlinear function approximators (neural networks) can lead to divergence and instability (the "deadly triad").
- **Requires a Small Number of discrete Actions**
- **Inability to Learn Stochastic Policies:** Needs to use ϵ-greedy policies for exploration.

Policy gradient methods

Advantages

- **Stochastic Policies:** Stochastic policies can automatically explore.
- **Smoother Convergence:** Small changes in policy parameters typically lead to smooth, incremental changes in the behavior of the policy, offering better theoretical convergence properties.
- **Simpler Policy Representation:** Does not need a large table.
- **Continuous and High-Dimensional Action Spaces:** Can naturally output probability distributions over continuous actions, making them ideal for robotics and fine control.

Disadvantages

- **Local Optima:** Generally, converges only to local optima.
- **Sample Inefficiency:** Pure policy gradients typically require a large number of interactions with the environment.
- **Off-Policy Difficulty:** They are harder to formulate and train in off-policy settings (some modern actor-critic methods help mitigate this).

What do you need to know?



- Learning a **parameterized policy** has many advantages:
  - Learn probabilities for taking an action (**stochastic policies**).
  - The policy starts typically soft and **automatically explores** and later can approximate the optimal discrete policy.
  - **The policy gradient theorem** provides strong performance bounds for gradient ascent, typically converging to a local optimum.
  - Can handle **continuous action** spaces.

- Important methods:
  - **Soft-max in action preferences** is a very popular choice for parameterized policies with a small set of discrete actions.
  - **REINFORCE:** Use MC to learn the policy parameters.
  - **Actor-Critic methods:** Estimate policy and value function parameters together using TD. Reduces REINFORCE's variance and leads to more stable and faster learning.

- **Policy gradient methods vs. value function approximation**:
  Each has a different set of strengths and weaknesses.
  The best choice depends on the problem.
- **Actor-Critic methods** are a hybrid of both.

